

Technometrics

ISSN: 0040-1706 (Print) 1537-2723 (Online) Journal homepage: www.tandfonline.com/journals/utch20

Cokriging-Based Sequential Design Strategies Using Fast Cross-Validation Techniques for Multi-Fidelity Computer Codes

Loic Le Gratiet & Claire Cannamela

To cite this article: Loic Le Gratiet & Claire Cannamela (2015) Cokriging-Based Sequential Design Strategies Using Fast Cross-Validation Techniques for Multi-Fidelity Computer Codes, Technometrics, 57:3, 418-427, DOI: 10.1080/00401706.2014.928233

To link to this article: https://doi.org/10.1080/00401706.2014.928233

View supplementary material

Published online: 17 Aug 2015.

Submit your article to this journal

Article views: 1662

View related articles

View Crossmark data

Citing articles: 30 View citing articles

Full Terms & Conditions of access and use can be found at
https://www.tandfonline.com/action/journalInformation?journalCode=utch20

Supplementary materials for this article are available online. Please go to http://www.tandfonline.com/r/TECH

Cokriging-Based Sequential Design Strategies Using Fast Cross-Validation Techniques for Multi-Fidelity Computer Codes

Loic Le GRATIET

Laboratoire de Probabilités et Modéles Aléatoires, Université Paris Diderot
Paris Cedex 13, France
(*loic.legratiet@gmail.com*)

Claire CANNAMELA

CEA, DAM, DIF
Arpajon, France
(*claire.cannamela@cea.fr*)

Cokriging-based surrogate models have become popular in recent decades to approximate a computer code output from a few simulations using both coarse and more complex versions of the code. In practical applications, it is common to sequentially add new simulations to obtain more accurate approximations. We propose a method of cokriging-based sequential design, which combines both the error evaluation provided by the cokriging model and the observed errors of a leave-one-out cross-validation procedure. This method is proposed in two versions, the first one selects points one at a time. The second one allows us to parallelize the simulations and to add several design points at a time. The main advantage of the suggested strategies is that at a new design point they choose which code versions should be simulated (i.e., the complex code or one of its fast approximations). A multifidelity application is used to illustrate the efficiency of the proposed approaches. In this example, the accurate code is a two-dimensional finite element model and the less accurate one is a one-dimensional approximation of the system. This article has supplementary material online.

KEY WORDS: Multifidelity codes; Resource allocations.

1. INTRODUCTION

Kriging-based surrogate models have become very popular in recent decades in the design and analysis of computer experiments (see Sacks et al. 1989; Santner, Williams, and Notz 2003; Ankenman, Nelson, and Staum 2010). Usually, in real applications, two stages are performed to approximate a computer code with a kriging model. The first one consists of building a kriging model from simulations coming from an initial experimental design set. The second stage consists of adding simulations sequentially at new design points that augment the initial design. The selection of the new points is usually based on criteria to improve the global accuracy of the model and this will be our goal. Sequential kriging has also been widely used in optimization (see Jones, Schonlau, and Welch 1998) and to estimate probabilities of failure (see Bect et al. 2012).

Kriging has recently been extended to allow for the use of coarse versions of a complex computer code to improve the accuracy of its approximation. These so-called multifidelity surrogate models have become of growing interest. Craig et al. (1998) proposed a linear regression formulation; this was improved by Cumming and Goldstein (2009) through a Bayes' linear formulation. The first multifidelity model using cokriging was suggested by Kennedy and O'Hagan (2000). Several works dealing with this model have been developed (see Kennedy and O'Hagan 2001; Higdon et al. 2004; Reese et al. 2004; Qian and Wu 2008; Le Gratiet 2013; Le Gratiet and Garnier 2014; Tuo, Wu, and Yu in press). Defining sequential design strategies in a multifidelity framework is also of interest and is still an open problem. A method based on nested Latin hypercube designs is suggested in Xiong et al. (2013). However, the method of Xiong et al. does not allow for adding a few simulations (e.g., it cannot perform a one step at-a-time sequential design) and it does not take into account the accuracy of the coarse code versions and the computational time ratios between two code levels.

Cokriging models are powerful tools to enrich an experimental design set since the predictor's mean squared error (MSE)—also called cokriging variance—has a closed form expression. However, the model MSE can suffer from an important flaw when the accuracy of the cokriging model is not homogenous over the input parameter space. Indeed, the cokriging variance is determined by the distances between prediction and design points but not by the model errors. To fix this important flaw, we can use an external test set. Nevertheless, in a complex computer code framework, it could be too expensive to consider an external test set and cross-validation (CV)-based criteria are more appropriate. As an illustration, Kleijnen and van Beers (2004) combined bootstrapping and CV to evaluate the predictor's MSE. Although this method improves the classical approach, it still does not take into account the model errors. We note that a strength of the method proposed by Kleijnen and van Beers (2004) is that it can be applied to any type of surrogate model. In this article, we suggest an original criterion combining the cokriging variance and the leave-one-out (LOO)

© 2015 American Statistical Association and the American Society for Quality
TECHNOMETRICS, AUGUST 2015, VOL. 57, NO. 3
DOI: 10.1080/00401706.2014.928233
Color versions of one or more of the figures in the article can be found online at *www.tandfonline.com/r/tech*.

CV errors. Furthermore, this criterion is presented for kriging model (see supplementary Section 1) and a numerical study is conducted for them (see supplementary Section 2). In particular, a comparison with other classical criteria is performed, which shows its relevance especially for functions with heterogenous variations.

The focus of this article is on sequential design for the cokriging model. We adapt strategies suggested for kriging models to multifidelity cokriging models. The strength of the proposed strategies is that they not only provide the new points at which to perform new simulations but they also determine which version of code to simulate. These criteria take into account the computational time ratios between code versions. They are based on a proxy of the integrated mean squared error (IMSE) reduction and on an original result giving the contribution of each code to the total variance of the model. We note that sequential design in a multifidelity framework has also been applied for optimization purposes (see Huang et al. 2006; Forrester, Sobester, and Keane 2007).

This article is organized as follows. First, we present the proposed cokriging multifidelity model (Section 2) and sequential design strategies (Section 3). Second, we apply the sequential cokriging approach to a mechanical example (Section 4).

2. MULTIFIDELITY COKRIGING MODEL

A computer code can often be run at different levels of accuracy. In such a case, it can be effective to use low-accuracy versions of it to improve its approximation. For a given computational budget, cokriging models are well suited to build such multifidelity surrogate models (see Kennedy and O'Hagan 2000; Qian and Wu 2008; Le Gratiet 2013; Le Gratiet and Garnier 2014). In this section, we present the considered multifidelity cokriging models.

Let us suppose that we want to approximate the output of a computer code $y^s(x)$, $x \in Q \subset \mathbb{R}^d$ (Q compact) and that coarse versions of this code $(y^l(x))_{l=1,\ldots,s-1}$ are available. These codes are sorted by order of fidelity from the less accurate $y^1(x)$ to the most accurate $y^{s-1}(x)$. All code levels are modeled by Gaussian processes $(Y^l(x))_{l=1,\ldots,s}$, which satisfy the following relations for $l = 2, \ldots, s$ (see Le Gratiet and Garnier 2014):

$$\begin{cases} Y^l(x) = \rho_{l-1}\tilde{Y}^{l-1}(x) + \delta^l(x) \\ \tilde{Y}^{l-1}(x) \perp \delta^l(x) \\ \tilde{Y}^{l-1}(x) \sim [Y^{l-1}(x)|\mathbf{Y}^{(l-1)} = \mathbf{y}^{(l-1)}] \end{cases}, \tag{1}$$

where $\mathbf{Y}^{(l)} = (\mathbf{Y}^1, \ldots, \mathbf{Y}^l)$ with $\mathbf{Y}^l = Y^l(\mathbf{D}^l)$, $\mathbf{y}^{(l)} = (\mathbf{y}^1, \ldots, \mathbf{y}^l)$ with $\mathbf{y}^l = y^l(\mathbf{D}^l)$ and $(\mathbf{D}^l)_{l=1,\ldots,s}$ are the experimental design sets at level l with n_l points and such that $\mathbf{D}^s \subseteq \mathbf{D}^{s-1} \subseteq \cdots \subseteq \mathbf{D}^1$. We note that the nested property is not necessary to build the model but allows for efficient parameter estimations. Furthermore, conditioning on parameters β_l and σ_l^2, $\delta^l(x)$ is a Gaussian process of mean $\mathbf{f}_l'(x)\beta_l$ with $\mathbf{f}_l'(x) = (f_1^l(x), \ldots, f_{p_l}^l(x))$ and covariance kernel $\sigma_l^2 r_l(x, \tilde{x})$. The functions $\mathbf{f}_l'(x) = (f_1^l(x), \ldots, f_{p_l}^l(x))$ are assumed to known. By convention $Y^1(x)$ has the same distribution as $\delta^1(x)$.

The multifidelity cokriging model at level $l = 2, \ldots, s$ is given by the following predictive distribution that integrates the posterior distributions of $(\rho_{l-1}\ \beta_l')'$, $l = 1, \ldots, s$

$$Y_{n_l}^l(x) \sim [Y^l(x)|\mathbf{Y}^{(l)} = \mathbf{y}^{(l)}].$$

We note that the distribution $[Y^l(x)|\mathbf{Y}^{(l)}(\omega) = \mathbf{y}^{(l)}]$, $\omega \in \Omega^l$ (Ω^l is the probability space where $\mathbf{Y}^{(l)}$ is defined), is denoted by $[Y^l(x)|\mathbf{Y}^{(l)} = \mathbf{y}^{(l)}]$ to simplify notation. As shown by Le Gratiet and Garnier (2014), by considering the Jeffrey's priors $p(\rho_{l-1}, \beta_l) \propto 1$ (p stands for the probability density function), the predictive mean of $Y_{n_l}^l(x)$ is

$$\begin{aligned} \mu_{n_l}^l(x) = {} & \hat{\rho}_{l-1}\mu_{n_{l-1}}^{l-1}(x) + \mathbf{f}_l'(x)\hat{\beta}_l \\ & + \mathbf{r}_l'(x)\mathbf{R}_l^{-1}(\mathbf{y}^l - \mathbf{F}_l\hat{\beta}_l - \hat{\rho}_{l-1}y^{l-1}(\mathbf{D}^l)), \end{aligned} \tag{2}$$

and its predictive variance is

$$\begin{aligned} k_{n_l}^l(x, \tilde{x}) = {} & \hat{\sigma}_{\rho_{l-1}}^2 k_{n_{l-1}}^{l-1}(x, \tilde{x}) \\ & + \sigma_l^2 \left(r_l(x, \tilde{x}) - \left(\mathbf{h}_l'(x)\mathbf{r}_l'(x)\right) \begin{pmatrix} 0 & \mathbf{H}_l' \\ \mathbf{H}_l & \mathbf{R}_l \end{pmatrix}^{-1} \begin{pmatrix} \mathbf{h}_l\ (\tilde{x}) \\ \mathbf{r}_l\ (\tilde{x}) \end{pmatrix} \right), \end{aligned} \tag{3}$$

where $\mathbf{H}_l = (y^{l-1}(\mathbf{D}^l)\mathbf{F}_l)$, $\mathbf{F}_l$ are the values of $\mathbf{f}_l'(x)$ at points in $\mathbf{D}^l$, $\mathbf{r}_l(x)$ is the correlation vector between $\mathbf{D}^l$ and x with respect to the correlation function $r_l(x, \tilde{x})$, $\mathbf{R}_l$ is the correlation matrix of $\mathbf{D}^l$ with respect to $r_l(x, \tilde{x})$, $\mathbf{h}_l'(x) = (\mu_{n_{l-1}}^{l-1}(x)\mathbf{f}_l'(x))$, $(\hat{\rho}_{l-1}\hat{\beta}_l')' = (\mathbf{H}_l'\mathbf{R}_l^{-1}\mathbf{H}_l)^{-1}\mathbf{H}_l'\mathbf{R}_l^{-1}\mathbf{y}^l$ is the mean of the posterior distribution of $(\rho_{l-1}\beta_l')'$ and $\hat{\sigma}_{\rho_{l-1}}^2 = \hat{\rho}_{l-1}^2 + [(\mathbf{H}_l'\mathbf{R}_l^{-1}\mathbf{H}_l)^{-1}]_{1,1}$ is the two-order moment of the posterior distribution of ρ_{l-1}. The notation $\mathbf{A}_{1,1}$ indicates the element $(1, 1)$ of the matrix $\mathbf{A}$ We note that the Jeffrey's prior is a noninformative prior distribution that reflects that we have no prior information about the parameters $(\rho_{l-1}\beta_l')'$.

The parameter $\{\sigma_l^2\}_{l=1,\ldots,s}$ are assumed to be known. In practice, they are estimated with a restricted maximum likelihood method. The corresponding estimate of σ_l^2 is given by

$$\begin{aligned} \sigma_l^2 = {} & \left(\mathbf{y}^l - \mathbf{H}_l\begin{pmatrix} \hat{\rho}_{l-1} \\ \hat{\beta}_l \end{pmatrix}\right)' \mathbf{R}_l^{-1} \left(\mathbf{y}^l - \mathbf{H}_l\begin{pmatrix} \hat{\rho}_{l-1} \\ \hat{\beta}_l \end{pmatrix}\right) \\ & \Big/ (n_l - p_l - 1), \end{aligned} \tag{4}$$

(see Le Gratiet and Garnier 2014).

Remark. The important property of this cokriging model is that its MSE (3) provides through the term $\hat{\sigma}_{\rho_{l-1}}^2 k_{n_{l-1}}^{l-1}$ the contribution of the code level $l-1$ to the total predictor's MSE. Therefore, it can allow us to determine which code level is worth being simulated at a new location x. Moreover, the nested property for the experimental design sets is necessary to extract those contributions.

3. SEQUENTIAL DESIGN FOR MULTIFIDELITY COKRIGING MODELS

We present in this section sequential design strategies in the multifidelity framework. It is important to notice that in a multifidelity framework, the search for the best locations at which to run the code is not the only objective. Indeed, once the best locations are determined, we also have to decide which code level to run. This will not only depend on the time ratios between the code levels but also on the contribution of each code level to the total predictor's MSE.

3.1 One-Point-At-a-Time Sequential Cokriging

Let us consider x_{new} the point solving the problem

$$x_{\text{new}} = \arg\max_x k^s_{n_s}(x, x).$$

Therefore, we want to compute a new simulation at a point where the predictor's MSE is maximal. Now, let us consider two successive code levels $l-1$ and l. The question of interest is to decide which of these two codes should be simulated.

First, using (3), we can deduce the contribution of each code levels to the predictor's MSE. Let us define the following notation for $l = 2, \ldots, s$:

$$\sigma^2_{\delta^l}(x) = \sigma_l^2 \left(1 - \left(\mathbf{h}_l'(x)\ \mathbf{r}_l'(x)\right) \begin{pmatrix} 0 & \mathbf{H}_l' \\ \mathbf{H}_l & \mathbf{R}_l \end{pmatrix}^{-1} \begin{pmatrix} \mathbf{h}_l(x) \\ \mathbf{r}_l(x) \end{pmatrix}\right),$$

and $\sigma^2_{\delta^1}(x) = k^1_{n_1}(x, x)$. Then, we have

$$k^l_{n_l}(x, x) = \sum_{j=1}^{l} \sigma^2_{\delta^j}(x) \prod_{k=j}^{l-1} \hat{\sigma}^2_{\rho_k}. \tag{4}$$

Let us consider that the kernels $\{(r_l(x, \tilde{x}; \theta_l)\}_{i=1,\ldots,s}$ are parameterized with $\{\theta_l\}_{l=1,\ldots,s}$, where $\{\theta_l\}_{l=1,\ldots,s}$ are the kernel characteristic length scales (see Rasmussen and Williams 2006, p. 83). The parameters $\{\theta_l\}_{l=1,\ldots,s}$ are assumed to be known. In practice, they are estimated with a restricted maximum likelihood method. The IMSE at level l is

$$\text{IMSE}^l = \int_Q k^l_{n_l}(x, x)\, dx.$$

Then, we can approximate the reduction of the IMSE after adding a new point x_{new} at level l as

$$\text{IMSE}^l_{\text{red}}(x_{\text{new}}) = \sum_{j=1}^{l} \sigma^2_{\delta^j}(x_{\text{new}}) \prod_{k=j}^{l-1} \hat{\sigma}^2_{\rho_k} \prod_{m=1}^{d} \theta_j^m, \tag{5}$$

with $\theta_l = (\theta_l^1, \ldots, \theta_l^d)$. Indeed, at each stage, $\sigma^2_{\delta^j}(x_{\text{new}}) \prod_{k=j}^{l-1} \hat{\sigma}^2_{\rho_k}$ represents the contribution of the bias $\delta^j(x)$ to the cokriging variance and $\prod_{m=1}^{d} \theta_j^m$ represents the volume of influence of x_{new} at level j. This criterion is justified by the fact that the reduction of $\bar{\sigma}^2_{\delta^l} = \int_Q \sigma^2_{\delta^l}(x)\, dx$ after adding a new point x_{new} has the same order of magnitude as $\sigma^2_{\delta^l}(x_{\text{new}})$ times the volume of influence $\prod_{m=1}^{d} \theta_l^m$ of x_{new}.

Now, let us consider that the ratio of computational times between the codes $y^l(x)$ and $y^{l-1}(x)$ defined as $B_{l/l-1}$. It is worth running the code $y^{l-1}(x)$ if $B_{l/l-1}\text{IMSE}^{l-1}_{\text{red}}(x_{\text{new}}) > \text{IMSE}^l_{\text{red}}(x_{\text{new}})$, that is, if the potential uncertainty reduction by running $B_{l/l-1}$ times $y^{l-1}(x)$ is greater than the reduction when we run one simulation on $y^l(x)$. From this criterion, we can formulate Algorithm 1 for a one-at-a-time sequential cokriging model taking into account both the computational ratios between the different code levels and the contribution of each level to the total cokriging variance.

Algorithm 1 evaluates for two successive code levels $l-1$ and l, which one should be simulated. It starts with the levels one and two, then two and three and so on. When it finds that the level $l-1$ is more promising than the level l (see remark below), it stops the loop and simulates x_{new} at code levels $y^1(x), \ldots, y^{l-1}(x)$. Since the loop is defined from level 1 to level s, it favors simulations at low code levels. Therefore, it will tend to learn the coarse code versions before learning the accurate ones. We note that during the loop of Algorithm 1, the parameters $\{\theta_l, \sigma_l^2\}_{l=1,\ldots,s}$ are not reestimated. They are reestimated after adding the new point x_{new} with a restricted maximum likelihood method. Moreover, the first test $\sigma^2_{\delta^l}(x_{\text{new}}) < \bar{\sigma}^2_{\delta^l}$ checks on average if the code level l at point x_{new} is worth being run. Then, the test $\text{IMSE}^{l-1}_{\text{red}}(x_{\text{new}})/\text{IMSE}^l_{\text{red}}(x_{\text{new}}) > 1/B_{l/l-1}$ evaluates which code levels between l and $l-1$ is the most promising. Finally, if we consider that the code level l is more promising than the code level $l-1$, we compare it to the following code level $l+1$. We note that Algorithm 1 is iterated until a prescribed accuracy is reached or the computational time budget is spent.

Algorithm 1 One-point-at-a-time sequential cokriging

```
Find x_new such that x_new = arg max_x k^s_{n_s}(x, x)
for l = 2, ..., s do
    if (σ²_{δ^l}(x_new) < σ̄²_{δ^l}) then
        Run y^{l-1}(x_new) end for
    else
        if (IMSE^{l-1}_red(x_new)/IMSE^l_red(x_new) > 1/B_{l/l-1}) then
            Run y^{l-1}(x_new) end for
        end if
    end if
end for
if (l = s) then
    Run y^l(x_new)
end if
```

Remark. In this work, we assume the monotonicity of the efficiency ratio as a function of the level l. However, one can decide to not consider this assumption and to evaluate all levels rather than stopping as soon as level $l-1$ is more promising than level l. In this case, the most promising code level will be the one minimizing the quantity

$$\prod_{i=l}^{s} B_{i+1/i}\text{IMSE}^i_{\text{red}}(x_{\text{new}}),$$

where by convention $B_{1+1/1} = 1$. It represents the potential uncertainty reduction at level l when for a computational budget corresponding to one simulation on the highest code level.

3.2 One-Point-At-a-Time Sequential Cokriging with Adjusted Predictor's MSE

An important flaw of the cokriging variance (as well as the kriging variance) is that it might not reflect the true model errors. We address this by adjusting the variance with the observed LOO-CV errors. The performance of the suggested criterion is studied in supplementary Section 2 where a comparison between different sequential design strategies is performed for the case $s = 1$ (i.e., for kriging models).

3.2.1 LOO-CV Equations for Multi-Fidelity Cokriging Models. First, we introduce some notation.

Notations. $\mathbf{A}_{i,i}$ is the ith element of the main diagonal of $\mathbf{A}$, $\mathbf{A}_i$ is the ith row of the matrix $\mathbf{A}$, $\mathbf{A}_{-i}$ is the matrix $\mathbf{A}$ without its ith row, $\mathbf{A}_{-i,i}$ is the ith column of $\mathbf{A}$ without its ith element,

$\mathbf{A}_{i,-i} = \mathbf{A}'_{-i,i}$ and $\mathbf{A}_{-i,-i}$ is the matrix $\mathbf{A}$ without the ith row and column.

For $l = 1, \ldots, s$ fixed, let us consider the multifidelity model (1) with $\mathbf{D}^l \subseteq \mathbf{D}^{l-1} \subseteq \cdots \subseteq \mathbf{D}^1$. We denote by i_j, the index of the element of $\mathbf{D}^j$ corresponding to the ith point x_i^l of $\mathbf{D}^l$ with $1 \leq j \leq l$. Then, if we denote by $\mu^l_{n_l,-i_l}(x)$ the predictive mean (2) built without the observations $(\mathbf{y}^1_{i_1}, \mathbf{y}^2_{i_2}, \ldots, \mathbf{y}^l_{i_l})$ and $\varepsilon_{\text{LOO-CV},l}(x_i^l) = \mathbf{y}^l_{i_l} - \mu^l_{n_l,-i_l}(x)$ the error of the LOO-CV prediction—called in this article LOO-CV error—(i.e., real value minus predicted value) at level l and point x_i^l, we have

$$\varepsilon_{\text{LOO-CV},l}\left(x_i^l\right) = \hat{\rho}_{l-1,i_l}\varepsilon_{\text{LOO-CV},l-1}\left(x_i^l\right) + \left[\mathbf{R}_l^{-1}\left(\mathbf{y}^l - \mathbf{H}_l\begin{pmatrix}\hat{\rho}_{l-1,i_l}\\ \hat{\boldsymbol{\beta}}_{l,i_l}\end{pmatrix}\right)\right]_{i_l} \Big/ \left[\mathbf{R}_l^{-1}\right]_{i_l,i_l}, \tag{6}$$

where $\hat{\rho}_{0,i_l} = 0$, $(\hat{\rho}_{l-1,i_l}\hat{\boldsymbol{\beta}}'_{l,i_l})' = (\mathbf{H}'_{l,-i_l}\mathbf{K}_l\mathbf{H}_{l,-i_l})^{-1}\mathbf{H}'_{l,-i_l}\mathbf{K}_l\mathbf{y}^l_{-i_l}$ and $\mathbf{K}_l = [\mathbf{R}_l^{-1}]_{-i_l,-i_l} - [\mathbf{R}_l^{-1}]_{-i_l,i_l}[\mathbf{R}_l^{-1}]_{i_l,-i_l}/[\mathbf{R}_l^{-1}]_{i_l,i_l}$. Therefore, the LOO-CV error at level l (6) is expressed as a function of the LOO-CV error at level $l-1$ adjusted with $\hat{\rho}_{l-1,i_l}$ plus a term representing the contribution to the LOO-CV error of the bias $\delta^l(x)$. The term $\hat{\rho}_{l-1,i_l}$ represents the adjustment coefficient estimated without the observation $\mathbf{y}^l_{i_l}$. We note that for the case $s = 1$, we obtain the equations given in Dubrule (1983) for the kriging case. The computational complexity of (6) is essentially determined by the matrix inversions of $(\mathbf{R}_j)_{j=1,\ldots,l}$, which have been already computed to obtain (2) and (3).

Furthermore, if we denote by $\sigma^2_{\text{LOO-CV},l}(x_i^l)$ the variance of the LOO-CV prediction—called in this article LOO-CV variance—(i.e., the predictive variance (3) evaluated at point x_i^l and built without the observations $(\mathbf{y}^1_{i_1}, \mathbf{y}^2_{i_2}, \ldots, \mathbf{y}^l_{i_l})$), we have

$$\sigma^2_{\text{LOO-CV},l}\left(x_i^l\right) = \hat{\sigma}^2_{\rho_{l-1,i_l}}\sigma^2_{\text{LOO-CV},l-1}\left(x_i^l\right) + \sigma^2_{l,-i_l}/\left[\mathbf{R}_l^{-1}\right]_{i_l,i_l} + \varsigma_l, \tag{7}$$

with $\hat{\sigma}^2_{\rho_{l-1,i_l}} = \hat{\rho}^2_{l-1,i_l} + [(\mathbf{H}'_{l,-i_l}\mathbf{K}_l\mathbf{H}_{l,-i_l})^{-1}]_{1,1}$, $\varsigma_l = \mathrm{u}_l^2(\mathbf{H}'_{l,-i_l}\mathbf{K}_l\mathbf{H}_{l,-i_l})^{-1}$, $\mathrm{u}_l = [\mathbf{R}_l^{-1}\mathbf{H}_l]_{i_l}/[\mathbf{R}_l^{-1}]_{i_l,i_l}$, and

$$\sigma^2_{l,-i_l} = \frac{\left(\mathbf{y}^l_{-i_l} - \mathbf{H}_{l,-i_l}\begin{pmatrix}\hat{\rho}_{l-1,i_j}\\ \hat{\boldsymbol{\beta}}_{l,i_j}\end{pmatrix}\right)' \mathbf{K}_s \left(\mathbf{y}^l_{-i_l} - \mathbf{H}_{l,-i_l}\begin{pmatrix}\hat{\rho}_{l-1,i_j}\\ \hat{\boldsymbol{\beta}}_{l,i_j}\end{pmatrix}\right)}{n_l - p_l - 2},$$

where $\mathbf{H}_{l,-i_l} = [y^{l-1}(\mathbf{D}^l_{-i_l}) \quad \mathbf{F}_{l,-i_l}]$. The LOO-CV variance at level l (7) is expressed as a function of the LOO-CV variance at level $l-1$ plus a term representing the contribution of the bias $\delta^l(x)$. Furthermore, the term $\hat{\sigma}^2_{\rho_{l-1,i_l}}$ corresponds to the second-order moment of the adjustment coefficient evaluated without the observation $\mathbf{y}^l_{i_l}$ and ς_l is a term representing the uncertainty due to the trend and the adjustment parameters at level l. We note that, as for (6), (7) does not require the matrix inversions of $(\mathbf{R}_j)_{j=1,\ldots,l}$.

Equations (6) and (7) provide closed form expressions for the LOO-CV errors and variances at level l. The proof of the presented equations is given in supplementary Section 3. From them, the LOO-CV equations are fast to compute—the complexity to obtain the LOO-CV predictive mean and variance for all $i = 1, \ldots, n_l$ is $\mathcal{O}(n_l \sum_{j=1}^l n_j^2)$ instead of $\mathcal{O}(n_l \sum_{j=1}^l n_j^3)$—and consequently they can be used in a sequential procedure with a low computational cost.

3.2.2 The Adjusted Cokriging Variance. From (6) and (7) we can adjust the cokriging variance $k^l_{n_l}(x,x)$ at each level $l = 1, \ldots, s$ as follows:

$$k^l_{n_l,\text{adj}}(x,x) = \sum_{j=1}^{l}\sigma^2_{\delta^j}(x)\prod_{k=j}^{l-1}\hat{\sigma}^2_{\rho_k} \times \left(1 + \sum_{i=1}^{n_j}\frac{\left(\varepsilon_{\text{LOO-CV},j}\left(x_i^j\right) - \hat{\rho}_{j-1,i_j}\varepsilon_{\text{LOO-CV},j-1,i_j}\left(x_i^j\right)\right)^2}{\sigma^2_{\text{LOO-CV},j}\left(x_i^j\right) - \hat{\sigma}^2_{\rho_{j-1},i_j}\sigma^2_{\text{LOO-CV},j-1,i_j}\left(x_i^j\right)}\mathbf{1}_{x\in V_i^j}\right) \tag{8}$$

where $\mathbf{1}$ stands for the indicator function and $(V_i^j)_{i=1,\ldots,n_j}$ are the Voronoï cells associated with the points $(x_i^j)_{i=1,\ldots,n_j}$ at level $j = 1, \ldots, s$: $V_i^j = \{x \in Q, \|x - x_i^j\| \leq \|x - x_k^j\|, \forall i \neq k, i, k = 1, \ldots, n_j\}$.

We use the convention $\hat{\rho}_0 = 0$ and $\hat{\sigma}^2_{\rho_0} = 0$. We note that $(\varepsilon_{\text{LOO-CV},j}(x_i^j) - \hat{\rho}_{j-1,i_j}\varepsilon_{\text{LOO-CV},j-1}(x_i^j))^2$ is the part of the LOO-CV squared error explained by the bias $\delta^j(x)$ and $\sigma^2_{\text{LOO-CV},j}(x_i^j) - \hat{\sigma}^2_{\rho_{j-1},i_j}\sigma^2_{\text{LOO-CV},j-1}(x_i^j)$ is the corresponding LOO-CV predictive variance. The principle of the adjusted variance (8) is to enlarge the predictive variance around the points where the LOO-CV errors are underestimated (i.e., when the LOO-CV squared errors are smaller than the predictive variances). For a given point, the domain where the variance is adjusted is defined by its Voronoi cell. Furthermore, the predictive variances are adjusted level by level since the LOO-CV errors can be poorly estimated just for a given level $j \in \{1, \ldots, l\}$. The implicit assumption is to consider that the LOO predictive error and variance at point x_i^l is a proxy for the actual error for all points in the Voronoi V_i^l cells of x_i^l.

We illustrate in Figure 1 the adjusted variance (8) for the case $s = 1$ (i.e., the kriging case, see supplementary Section 1) in a one-dimensional example. In this case, the adjusted variance is

$$k^1_{n_1,\text{adj}}(x,x) = \sigma^2_{\delta^1}(x)\left(1 + \sum_{i=1}^{n_1}\frac{\varepsilon_{\text{LOO-CV},1}\left(x_i^1\right)^2}{\sigma^2_{\text{LOO-CV},1}\left(x_i^1\right)}\mathbf{1}_{x\in V_i^1}\right).$$

The considered function is $f(x) = (\sin(7x) + \cos(14x))x^2\exp(-4x)$, $x \in [0,4]$. We use a kriging model with a 5/2-Matérn kernel with $\sigma^2 = 1 \times 10^{-3}$ and $\theta = 1$ (see Rasmussen and Williams 2006, p. 84) and the experimental design set is a regular grid of 8 points between 0 and 4. We see in Figure 1 that the kriging model is not accurate in the domain $[0,2]$ where the function variations are important and the adjusted kriging variance criterion focuses on that region.

As illustrated in Figure 1, the adjusted kriging variance allows for taking into account the LOO-CV error in a sequential procedure focusing on the large error domain. Nevertheless, it does not entirely fix the issue of the relevance of $k^1_{n_1}(x,x)$ to represent the model error. Indeed, our criterion enlarges the kriging variance around points where $k^1_{n_1}(x,x)$ is under-estimated but it does not reduce it at locations where it is over-estimated. However, it gives more information about the relevance of $\mu^1_{n_1}(x)$ (2) since it highlights the regions where it is not accurate.

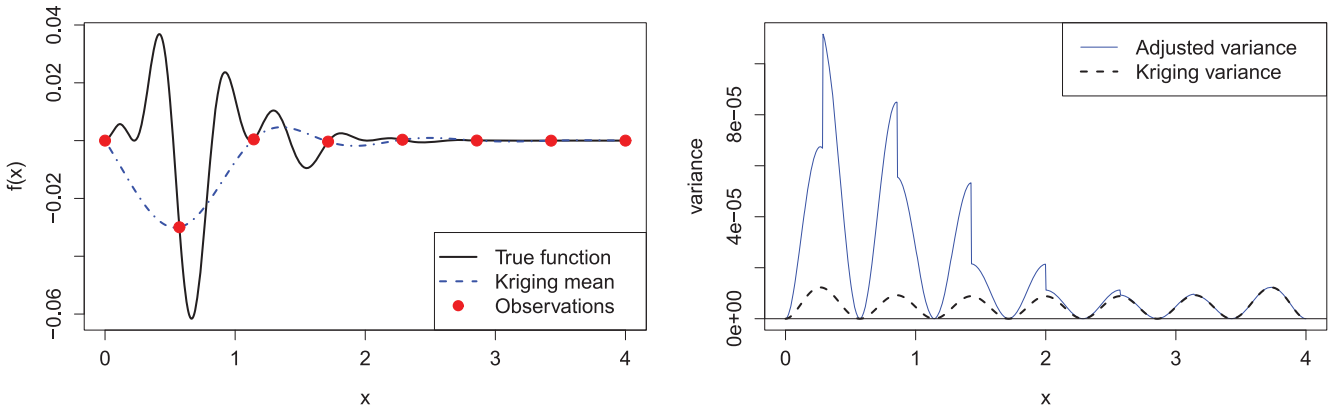


Figure 1. Illustration of the adjusted kriging variance in a one-dimensional example. We see that the kriging variance is enlarged at the domain where the function variations are important.

3.2.3 Sequential Procedure with Adjusted Variance. We can now consider in Algorithm 1 the point x_{new} solving the problem

$$x_{\text{new}} = \arg\max_x k^s_{n_s,\text{adj}}(x,x). \tag{9}$$

A sequential strategy with this criterion focuses on the regions of Q where the LOO-CV errors are the largest. We note that the normalization with the LOO-CV variances is important since it is not necessary to enlarge the predictor's MSE in the regions where it is well or over-estimated. An efficient method to solve the problem in (9) is to use an evolutionary algorithm coupled with a descent algorithm. To explore the different Voronoi cells, we can use a genetic algorithm or simply a Monte Carlo sample for low-dimensional problems (i.e., $d < 10$). Then, for a given Voronoi cell, the criterion to optimize is continuous. Therefore, we can solve the problem with classical optimization methods (direct or simplex methods). Furthermore, it is common to have covariance kernel such that the criterion is once or twice continuously differentiable. In this case, we can use a gradient method, a conjugate method or a Newton method (for the twice differentiable case). We note that it is not necessary to compute the Voronoi tessellation since the criterion only requires the Voronoi cell in which a given point $x \in Q \subset \mathbb{R}^d$ lies. This is computationally simple and cheap even for high dimension d.

Let us consider the quantity

$$\begin{aligned}&\text{IMSE}^l_{\text{red,adj}}(x_{\text{new}})\\ &= \sum_{j=1}^{l}\sigma^2_{\delta^j}(x_{\text{new}})\prod_{k=j}^{l-1}\hat\sigma^2_{\rho_k}\prod_{m=1}^{d}\theta_j^m\\ &\times\left(1+\sum_{i=1}^{n_j}\frac{\left(\varepsilon_{\text{LOO-CV},j}\left(x_i^j\right)-\hat\rho_{j-1,i_j}\varepsilon_{\text{LOO-CV},i-1}\left(x_i^j\right)\right)^2}{\sigma^2_{\text{LOO-CV},j}\left(x_i^j\right)-\hat\sigma^2_{\rho_{j-1},i_j}\sigma^2_{\text{LOO-CV},j-1}\left(x_i^j\right)}\mathbf{1}_{x\in V_i^j}\right),\end{aligned} \tag{10}$$

where $\hat\rho_{j-1,i_j}$ and $\hat\sigma^2_{\rho_{j-1},i_j}$ are given by (6) and (7). In (10), the variance $k^j_{n_j}(x,x)$ (4) in (5) is replaced with the adjusted variance.

To adapt Algorithm 1 with the adjusted variance, we just have to replace $\text{IMSE}^l_{\text{red}}(x)$ with $\text{IMSE}^l_{\text{red,adj}}(x)$ in Algorithm 1.

Remark. The sequential strategy (9) based on the adjusted variance (8) is suggested for the first time in this article. To emphasize its relevance, a comparison with other classical criteria is performed in supplementary Section 2. The comparison is made for the kriging case (i.e., $s = 1$) on academic examples and on the application presented in Section 4. This numerical study shows that the strategy (9) is more efficient for functions with heterogenous variations and performs as well as the best strategies when the variations are homogenous.

3.3 Batch Sequential Cokriging

In this section, we propose a batch sequential design strategy for multi-fidelity computer codes. Its principle is the following one. First, we select q^l new points for the code $y^l(x)$ (the method of selection is presented in Algorithm 2). Then, we consider these points as known for the codes $y^{l-1}(x),\dots,y^1(x)$ and we select q^{l-1} new points for the code $y^{l-1}(x)$ with the same method. We note that, as the variance (3) does not depend on the observations, we can compute the new cokriging variances without simulating $y^{l-1}(x),\dots,y^1(x)$ at the q^l new points. Finally, we repeat this procedure for all code levels from $y^{l-2}(x)$ to $y^1(x)$. At the end of the procedure, we have $\sum_{i=j}^l q^i$ new points at level j and we want to find the allocation $\{q^1,\dots,q^l\}$ giving the largest potential uncertainty reduction and under the constraint of a constant CPU time budget. We note the CPU time budget $T = \sum_{j=1}^l\sum_{i=j}^l q^i t^j$, where $(t^i)_{i=1,\dots,s}$ represents the CPU times of codes $(y^i(x))_{i=1,\dots,s}$. We assume that the computational time depends on the code level but not on the point x. Algorithm 2 presents the suggested q points at-a-time sequential cokriging.

Algorithm 2 details. In line 3, $k^l_{n_l}(x,x)$ comes from (3) and we generate samples with respect to the probability density function proportional to $k^l_{n_l}(x,x)$ with a Monte Carlo Markov chain (MCMC) technique (see Robert and Casella 2004). In particular, we use a Metropolis–Hasting (M-H) algorithm. In line 4, the N^l-clustering is performed from the N^l_{MCMC} samples generated in line 3 and using an N^l-means clustering technique (see MacQueen 1967). For the N^l-clustering procedure, we choose the value of N^l with respect to the criterion $N^l = \max_{N\geq q^l}\min_{x\in(C^l_i)_i}k^l_{n_l}(x,x)$, where $(C^l_i)_{i=1,\dots,N^l}$ are the cluster centers. This criterion prevents a cluster center C^l_i from being placed in a region where the kriging variance is close to zero. Furthermore, if the number of clusters is too large, the cluster centers get away from the modes of $k^l_{n_l}(x,x)$ and consequently the maximal value of $\min_{x\in(C^l_i)_i}k^l_{n_l}(x,x)$ decreases. Therefore, this criterion also prevents the number of clusters

Algorithm 2 $(q^i)_{i=1,\ldots,s}$ points at-a-time sequential cokriging

1. Set the budget $T > 0$ and the allocation $\{q^1, \ldots, q^l\}$ such that $\sum_{j=1}^{l} \sum_{i=j}^{l} q^i t^j = T$
2. Set $(N^i_{\text{MCMC}})_{i=1,\ldots,l}$ for the Metropolis–Hastings (M-H) procedures.
3. Generate N^l_{MCMC} samples with respect to $k^l_{n_l}(x, x)$.
4. Find the N^l cluster centers $(C^l_i)_{i=1,\ldots,N^l}$ such that $N^l = \max_{N \geq q^l} \min_{x \in (C^l_i)_i} k^l_{n_l}(x, x)$
5. Select from $(C^l_i)_{i=1,\ldots,N^l}$ the q^l points $(x^l_{\text{new,i}})_{i=1,\ldots,q_l}$ where $k^l_{n_l,\text{adj}}(x, x)$ is the largest.
6. **for** $m = l - 1, \ldots, 1$ **do**
7. Compute $k^m_{n_m+\sum_{i=m+1}^{l} q^i}(x, x)$ with the new points $((x^j_{\text{new,i}})_{i=1,\ldots,q^j})_{j=m+1,\ldots,l}$
8. Generate N^m_{MCMC} samples with respect to $k^m_{n_m+\sum_{i=m+1}^{l} q^i}(x, x)$.
9. Find the N^m cluster centers $(C^m_i)_{i=1,\ldots,N^m}$ such that $N^m = \max_{N \geq q^m} \min_{x \in (C^m_i)_i} k^m_{n_m+\sum_{i=m+1}^{l} q^i}(x, x)$
10. Select from $(C^m_i)_{i=1,\ldots,N^m}$ the q^m points $(x^m_{\text{new,i}})_{i=1,\ldots,q^m}$ where $k^m_{n_m+\sum_{i=m+1}^{l} q^i,\text{adj}}(x, x)$ is the largest.
11. **end for**

from being too large. We note the MCMC plus N^l-means procedure requires careful implementation and appropriate diagnostics. For the N^l-means procedure, we use the algorithm suggested by Hartigan and Wong (1979) with complexity $\mathcal{O}(N^l N^l_{\text{MCMC}})$. For the M-H procedure, we use the package "mcmc" (Martin et al. 2011). To avoid computational issues, one could instead extract the q^l-points from candidates generated with space-filling design techniques (see Fang, Li, and Sudjianto 2006). However, with this technique, the candidate points will not anymore be concentrated in the regions of high mean squared error and the method will be less efficient. In line 5, the N^l cluster centers are the candidate points from which we extract the q^l new points having the maximum adjusted variance $k^l_{n_l,\text{adj}}(x, x)$ (8). In the "For" loop, the same procedure is repeated for all code levels $m = l - 1, \ldots, 1$ except that we update the kriging variances $k^m_{n_m}(x, x)$ with the points added in level $m + 1, \ldots, l$ (since the experimental design sets must be nested).

Therefore, in Algorithm 2, $k^m_{n_m+\sum_{i=m+1}^{l} q^i}(x, x)$ corresponds to the kernel of the random process $Y^m_{n_m}(x)$ conditioned by the observations at points $((x^j_{\text{new,i}})_{i=1,\ldots,q^j})_{j=m+1,\ldots,l}$ when the parameters $(\sigma^2_i)_{i=m+1,\ldots,l}$ and $(\theta_i)_{i=m+1,\ldots,l}$ are considered as known (i.e., this corresponds to a liar method). Furthermore, $k^m_{n_m+\sum_{i=m+1}^{l} q^i,\text{adj}}(x, x)$ corresponds to the predictor's variance $k^m_{n_m+\sum_{i=m+1}^{l} q^i}(x, x)$ adjusted with the LOO-CV errors and variances. For the adjustment procedure, we do not consider the points $((x^j_{\text{new,i}})_{i=1,\ldots,q^j})_{j=m+1,\ldots,l}$. Indeed, the LOO-CV errors and variances are only available in the initial points $(x^j_i)_{i=1,\ldots,n_j\, j=1,\ldots,l}$. We note that for the M-H procedures, we use a Gaussian proposal distribution with a standard deviation such that the acceptance rate is around 30% (following the recommendation of Roberts, Gelman, and Gilks 1997) .

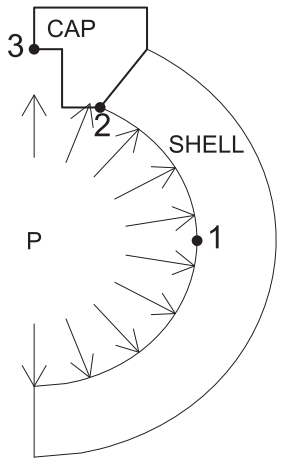


Figure 2. Scheme of the spherical tank under pressure.

Furthermore, let us consider the quantity

$$\text{IMSE}_{\text{red,q}} = \sum_{m=1}^{l} \sum_{j=m}^{l} \sum_{r=1}^{q^j} \sigma^2_{\delta^m}\left(x^j_{\text{new},r}\right) \prod_{k=m}^{l-1} \hat{\sigma}^2_{\rho_k} \prod_{k=1}^{d} \theta^k_m.$$

It corresponds to the potential IMSE reduction when we consider the allocation $\{q^1, \ldots, q^l\}$. We note that $\text{IMSE}_{\text{red},q}$ is a proxy on the IMSE reduction when we add $((x^m_{\text{new,i}})_{i=1,\ldots,q_m})_{m=1,\ldots,l}$ at code levels $(y^m(x))_{m=1,\ldots,l}$.

We consider the allocation that solves the optimization problem

$$\{q^1, \ldots, q^l\} = \arg \max_{\{q^1,\ldots,q^l\}} \text{IMSE}_{\text{red,q}} \text{ such that } \sum_{j=1}^{l} \sum_{i=j}^{l} q^i t^j = T,$$

that is, we look for the allocation leading the maximal uncertainty reduction. This optimization problem is very complex to solve and a suboptimal solution will be often considered. Nevertheless, when the number of code levels and the budget T are small (e.g., $s = 2$ in our application), an exhaustive exploration of the allocation $\{q^1, \ldots, q^l\}$ can be performed. We are in that case in the presented application .

In practical application, Algorithm 2 is repeated until we reach a prescribed precision or the computational time budget is exhausted.

Remark. This strategy is illustrated in supplementary Section 2 for the case $s = 1$ with an application solving a spherical tank under internal pressure. In particular, it is compared with classical strategies using a "liar" method (see supplementary Section 1.1.2). This comparison emphasizes that the MCMC+N-means procedure is worthwhile for a batch sequential kriging.

4. SPHERICAL TANK UNDER INTERNAL PRESSURE EXAMPLE

In this section, we deal with an example concerning a spherical tank under internal pressure. The physical system is represented in Figure 2. We are interested in the von Mises stresses on the three points labeled in Figure 2. Indeed, we want to prevent material from yielding, which occurs when the von Mises stress reaches the critical yield strength.

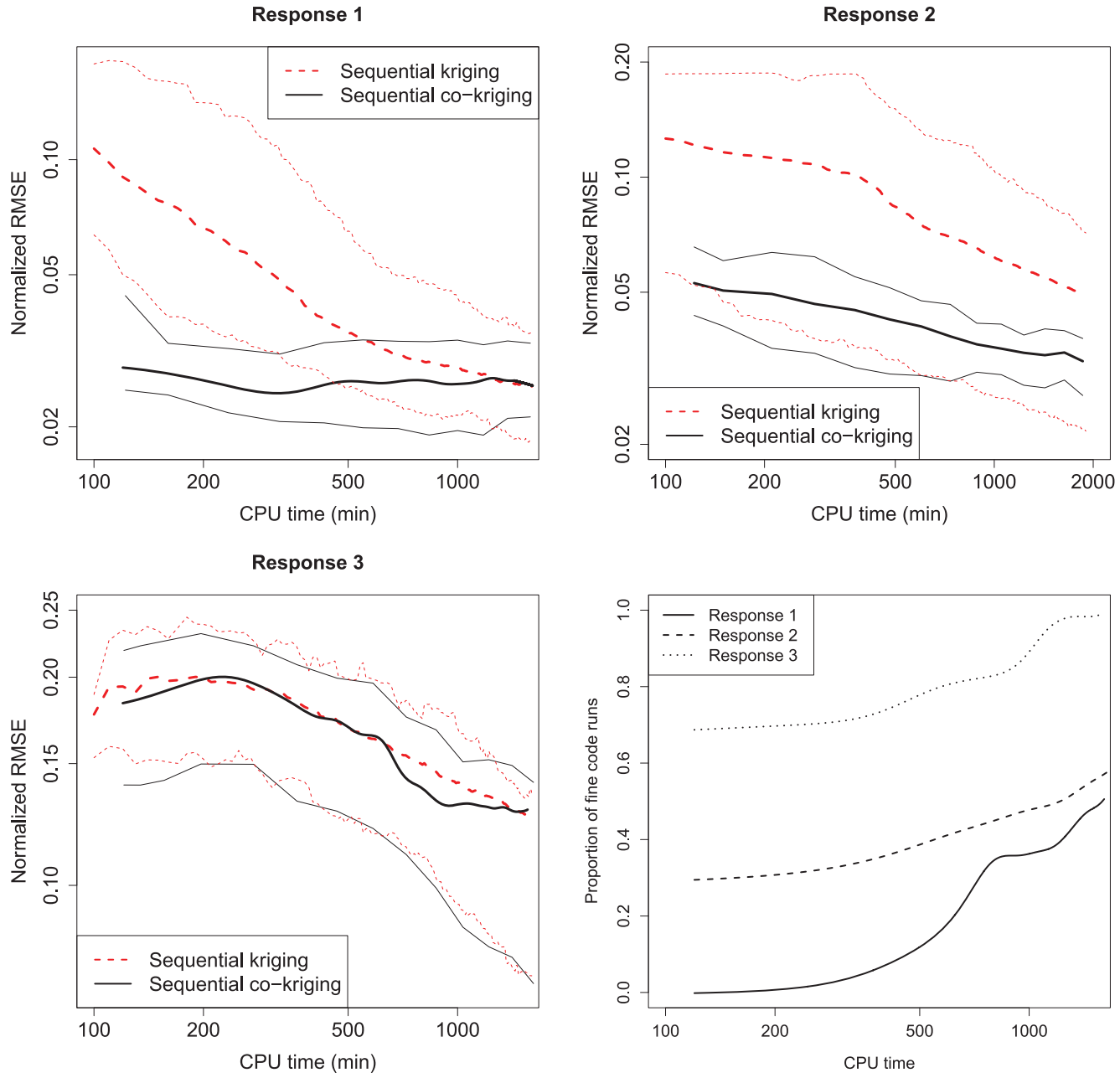


Figure 3. Comparison between one-point-at-a-time sequential kriging and cokriging on the response 1, 2, and 3 of the spherical tank example with respect to the criterion (9). The thin lines represent the quantiles of probabilities 10% and 90% of the Normalized RMSE. Figure on the bottom right represents the proportion of runs allocated to the accurate code.

The system depends on the following parameters:

P (MPa) $\in [30, 50]$:	the value of the internal pressure.
R_{int} (mm) $\in [1500, 2500]$:	the length of the internal radius of the shell.
T_{shell} (mm) $\in [300, 500]$:	the thickness of the shell.
T_{cap} (mm) $\in [100, 300]$:	the thickness of the cap.
E_{shell} (GPa) $\in [63, 77]$:	the Young's modulus of the shell material.
E_{cap} (GPa) $\in [189, 231]$:	the Young's modulus of the cap material.
$\sigma_{y,\text{shell}}$ (MPa) $\in [200, 300]$:	the yield stress of the shell material.
$\sigma_{y,\text{cap}}$ (MPa) $\in [400, 800]$:	the yield stress of the cap material.

The accurate code output $y^2(x)$ is the value of the von Mises stress provided by an Aster finite elements code (*http://www.code-aster.org*) modeling the system presented in Figure 2. We use the notation $x = (P, R_{\text{int}}, T_{\text{shell}}, T_{\text{cap}}, E_{\text{shell}}, E_{\text{cap}}, \sigma_{y,\text{shell}}, \sigma_{y,\text{cap}})$. Therefore, the input dimension of $y^2(x)$ and $y^1(x)$ is eight. We note that the material properties of the shell correspond to high-quality aluminum and the ones of the cap correspond to steel from conventional to high quality. The coarse code output $y^1(x)$ is the value of the von Mises stress given by the one-dimensional simplification of the tank (11) (corresponding to a perfect spherical tank under pressure, i.e., without cap),

$$y^1(x) = \frac{3}{2} \frac{(R_{\text{int}} + T_{\text{shell}})^3}{(R_{\text{int}} + T_{\text{shell}})^3 - R_{\text{int}}^3} P. \tag{11}$$

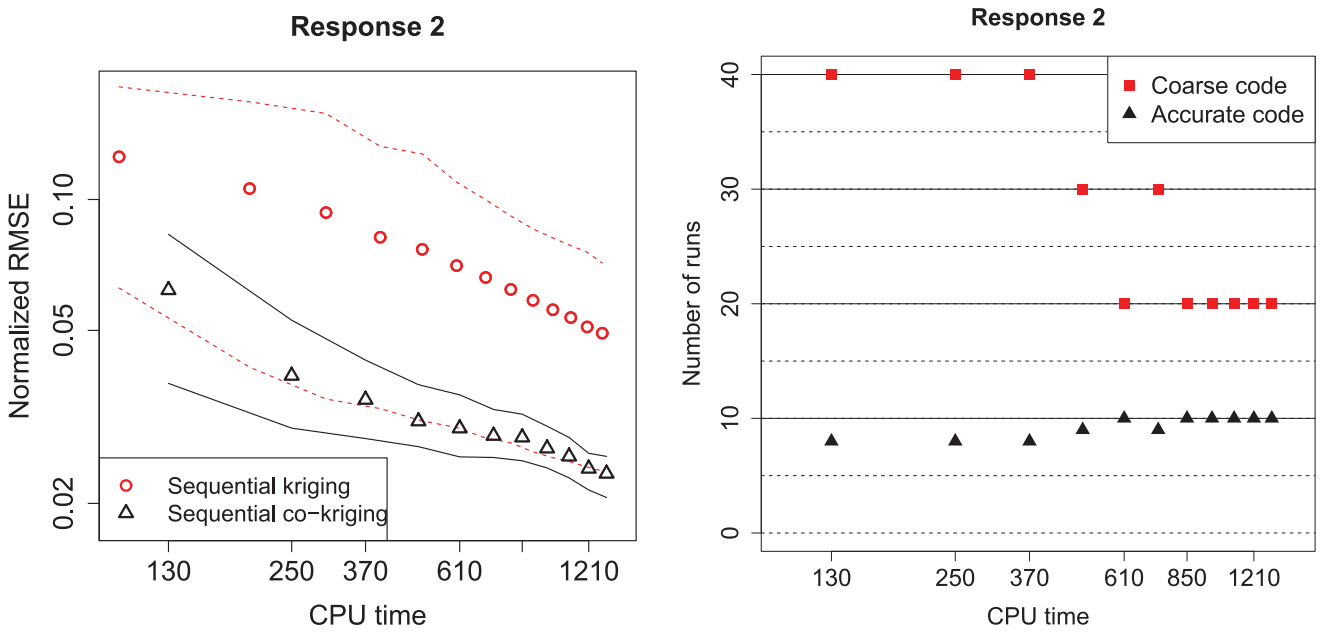


Figure 4. Comparison between $q = 10$-points-at-a-time sequential kriging and (q^1, q^2)-points-at-a-time sequential cokriging. The solid lines represent the quantiles of probabilities 10% and 90% of the Normalized RMSE for the sequential cokriging and the dashed ones represent it for the sequential kriging. On the right, the squares represent the median number of runs for the coarse code during the sequential cokriging and the triangles represent it for the accurate code.

According to (11), the actual input dimension of $y^1(x)$ is three (it depends only on P, R_{int}, and T_{shell}) while a sensitivity analysis performed with a Sobol decomposition method indicates that the accurate code depends essentially on four parameters (P, R_{int}, T_{shell}, and T_{cap}). Furthermore, the response is very smooth. Therefore, only a few points are necessary to accurately predict the output of the code. For these reasons, we can start the sequential strategies from an initial experimental design set with only 10 points.

Thus, for the different comparisons, we use an S-optimal LHS design $\mathbf{D}^2$ of 10 points for the code $y^2(x)$. The S-optimality maximizes the mean distance from each design point to the other points in the design. For the coarse code $y^1(x)$, we start with a design $\mathbf{D}^1$ of 20 points. It is created with the following procedure. First, we create an S-Optimal design $\tilde{\mathbf{D}}^1$ of 20 points. Second, we remove from $\tilde{\mathbf{D}}^1$ the 10 points that are the closest to those of $\mathbf{D}^2$. Finally, $\mathbf{D}^1$ is the concatenation of $\mathbf{D}^2$ and $\tilde{\mathbf{D}}^1$ (this procedure ensures the nested property $\mathbf{D}^2 \subset \mathbf{D}^1$). We note that the CPU time is around 1 min for the accurate code and 10^{-8} sec for the coarse code. Nevertheless, to be in a more realistic case, we consider that the CPU time ratio between $y^2(x)$ and $y^1(x)$ to be $B_{2/1} = 10$. Furthermore, each sequential procedure is performed with 40 different initial design sets. Then, the mean and the quantiles of probabilities 90% and 10% of the empirical Normalized MSE are computed from a test set composed of 1000 points uniformly spread on the input parameter space. Finally, for the M-H procedure, we use a Gaussian proposal distribution such that the acceptance rate is around 30% and we set $N_{\text{MCMC}} = 50{,}000$ (we use 5000 samples for the burn-in procedure of the M-H method, see Robert and Casella 2004). For the M-H procedure, we use the package mcmc. We note that after each point addition, the parameters $\{\theta_l, \sigma_l^2\}_{l=1,\ldots,s}$ of the kriging or cokriging models are reestimated with a restricted maximum likelihood method and that 5/2-Matérn kernels are used for all models.

In the forthcoming developments, the response $i = 1, 2, 3$ refers to the value of the von Mises stress at point i in Figure 2.

Comparison between kriging and cokriging sequential analysis. In the remainder of this section, we compare the sequential kriging strategy (i.e., with $s = 1$ and using only observations of $y^2(x)$) and the sequential cokriging with respect to the criterion presented in (9). Figure 3 gives the convergence of the empirical normalized root mean squared error Norm RMSE,

$$\text{Norm RMSE} = \frac{\sqrt{\sum_{i=1}^{n_{\text{test}}} \left(y_{\text{real}}\left(x_{\text{test}}^i\right) - y_{\text{pred}}\left(x_{\text{test}}^i\right)\right)^2 / n_{\text{test}}}}{\max_{x \in \mathbf{D}_{\text{test}}} y_{\text{real}}(x) - \min_{x \in \mathbf{D}_{\text{test}}} y_{\text{real}}(x)},$$

for the responses 1, 2, and 3.

We see that for response 1 sequential cokriging performs better than sequential kriging. Furthermore, in the early steps of the sequential method, the proportion of runs for the accurate code is very low. Indeed, the coarse code and the accurate code are extremely correlated for this response (around 99%) and thus, during the sequential strategy, the bias between the two codes is well estimated. Then, when the coarse code is well approximated, the sequential strategy starts to run the accurate one (for a CPU time around 500).

For response 2, the correlation between the coarse and the accurate code is around 80%. Therefore, the proportion of runs for the accurate code determined by the sequential strategy is more important than in the response 2. Furthermore, we see that this proportion increases with the CPU time. It means that the

sequential cokriging improves the approximation of the coarse code at the beginning of the procedure and then focuses on the accurate code. As a result, we see that the sequential cokriging strategy is substantially better than the kriging one.

For response 3, the coarse code is weakly correlated with the accurate code (around 45%). This is because that the coarse code models the von Mises stress in a perfect spherical tank whereas response 3 corresponds to the one in the cap. Figure 3 shows that in this case, the sequential cokriging model manages to determine that the coarse code is not worth being simulated. Indeed, the proportion of runs for the accurate code is very high. Furthermore, it shows that the cokriging sequential design performs as well as the kriging one when the coarse code is noninformative.

Finally, Figure 4 shows the efficiency of the (q^1, q^2) at-a-time sequential cokriging. We set in algorithm 2 that $T = q^1 + q^2 + 10q^2 = 120$ where the CPU time of the coarse code is 1 and the one of the accurate code is 10. For the sequential kriging, we use a $q = 10$-at-a-time sequential procedure. Furthermore, Figure 4 shows that at the beginning of the procedure, the sequential cokriging focuses on the approximation of the coarse code whereas at the end the number of runs for the accurate code is maximal. We note that the allocation of runs for the accurate code in Figure 4 agrees with the proportion of runs given in Figure 3.

The results of the sequential cokriging on the different responses show that the criterion suggested in Section 3 performs very well. Indeed, it is always better than the sequential kriging when the coarse code is informative and its performance is equivalent to it when the coarse code is not useful. Furthermore, the different proportions of runs for the accurate code emphasize that the criterion accurately determines the contribution of each code to the total model error and the optimal run allocation between the accurate and the coarse codes.

5. CONCLUSION

This article deals with sequential strategies for multifidelity cokriging models. This model is used to approximate the output of a complex computer code using coarse versions of it.

We have presented a sequential design strategy for the cokriging model and we have suggested a criterion based on the leave-one-out cross-validation errors. This criterion has allowed us to set the new observations at locations where the model error is important. The examples presented in the supplementary material highlight the efficiency of the suggested criterion. Indeed, for functions with heterogenous variations over the input parameter space, it provides results significantly better than classical criteria. For functions with homogenous variations, its performance is equivalent to them. We have also emphasized the performance of the suggested criterion on a real application. Furthermore, we show in the application that when the simulations can be performed in parallel, our method can be very efficient.

Then, we have shown in the application that performing a multifidelity sequential cokriging is worthwhile when the coarse code versions are informative (i.e., highly correlated with the accurate code). Furthermore, a strength of the proposed approach is that it performs as well as sequential kriging when the coarse code versions are not informative. In fact, the proposed sequential strategies take into account the contribution of each code to the total predictor's mean squared errors and they determine the best run allocation between accurate and coarse code versions given a CPU time budget.

SUPPLEMENTARY MATERIALS

Additional results and empirical study: Proofs of Equations (17)– (19) and a simulation study comparing different kriging-based simulation strategies. The relevance of the suggested adjusted variance and the MCMC+N-means procedure is demonstrated (pdf file).

Code: A zip file containing code used in the example in Section 4 and the supplementary file.

ACKNOWLEDGEMENTS

We particularly thank Professor Josselin Garnier for his constructive suggestions and helpful comments. We are also grateful to Gilles Defaux of the CEA—the French atomic energy authority—for providing the data for the application. All the numerical tests have been performed within the R environment, by using the DiceKriging (Roustant et al. 2012), MuFiCokriging (Le Gratiet 2013), rgenoud (Mebane and Sekhon 2011), and mcmc (Martin et al. 2011) packages.

[Received October 2012. Revised April 2014.]

REFERENCES

Ankenman, B., Nelson, B. L., and Staum, J. (2010), "Stochastic Kriging for Simulation Metamodeling," *Operations Research*, 58, 371–382. [418]

Bect, J., Ginsbourger, D., Li, L., Picheny, V., and Vazquez, E. (2012), "Sequential Design of Computer Experiments for the Estimation of a Probability of Failure," *Statistics and Computing*, 22, 773–793. [418]

Craig, P. S., Goldstein, M., Seheult, A. H., and Smith, J. A. (1998), "Constructing Partial Prior Specifications for Models of Complex Physical Systems," *Journal of the Royal Statistical Society*, Series D, 47, 37–53. [418]

Cumming, J. A., and Goldstein, M. (2009), "Small Sample Bayesian Designs for Complex High-dimensional Models Based on Information Gained Using Fast Approximations," *Technometrics*, 51, 377–388. [418]

Dubrule, O. (1983), "Cross Validation of Kriging in a Unique Neighborhood," *Journal of the International Association for Mathematical Geology*, 15, 687–699. [421]

Fang, K.-T., Li, R., and Sudjianto, A. (2006), *Design and Modeling for Computer Experiments*, London: Chapman & Hall—Computer Science and Data Analysis Series. [423]

Forrester, A. I. J., Sobester, A., and Keane, A. J. (2007), "Multifidelity Optimization Via Surrogate Modelling," *Proceedings of the Royal Society A*, 463, 3251–3269. [419]

Hartigan, J. A., and Wong, M. A. (1979), "Algorithm as 136: A k-means Clustering Algorithm," *Journal of the Royal Statistical Society*, Series C, 28, 100–108. [423]

Higdon, D., Kennedy, M., Cavendish, J. C., Cafeo, J. A., and Ryne, R. D. (2004), "Combining Field Data and Computer Simulation for Calibration and Prediction," *SIAM Journal on Scientific Computing*, 26, 448–466. [418]

Huang, D., Allen, T. T., Notz, W. I., and Miller, R. A. (2006), "Sequential Kriging Optimization Using Multiple-Fidelity Evaluations," *Structural and Multidisciplinary Optimization*, 32, 369–382. [419]

Jones, D. R., Schonlau, M., and Welch, W. J. (1998), "Efficient Global Optimization of Expensive Black-Box Functions," *Journal of Global Optimization*, 13, 455–492. [418]

Kennedy, M. C., and O'Hagan, A. (2000), "Predicting the Output from a Complex Computer Code When Fast Approximations are Available," *Biometrika*, 87, 1–13. [418,419]

——— (2001), "Bayesian Calibration of Computer Models," *Journal of the Royal Statistical Society*, Series B, 63, 425–464. [418]

Kleijnen, J., and van Beers, W. (2004), "Application-Driven Sequential Designs for Simulation Experiments: Kriging Metamodelling," *Journal of the Operational Research Society*, 55, 876–883. [418]

Le Gratiet, L. (2013), "Bayesian Analysis of Hierarchical Multifidelity Codes," *SIAM/ASA Journal on Uncertainty Quantification*, 1, 244–269. [418,419,426]

Le Gratiet, L., and Garnier, J. (2014), "Recursive Co-kriging Model for Design of Computer Experiments With Multiple Levels of Fidelity," *International Journal for Uncertainty Quantification*, 4, 365–386. [418,419]

MacQueen, J. (1967), "Some Methods for Classification and Analysis of Multivariate Observations," in *Proceedings of the 5th Berkeley Symposium on Mathematical Statistics & Probability* (Vol. 1), Berkeley, CA: University of California Press, pp. 281–297. [422]

Martin, A. D., Quinn, K. M., and Park, J. H. (2011), "MCMCpack: Markov Chain Monte Carlo in R," *Journal of Statistical Software*, 42, 22, 1–21. [423,426]

Mebane, W. R., Jr, and Sekhon, J. S., (2011), "Genetic Optimization Using Derivatives: The Rgenoud Package for R," *Journal of Statistical Software*, 42, 1–26. [426]

Qian, P. Z. G., and Wu, J. C. F. (2008), "Bayesian Hierarchical Modeling for Integrating Low-Accuracy and High-Accuracy Experiments," *Technometrics*, 50, 192–204. [418,419]

Rasmussen, C. E., and Williams, C. K. I. (2006), *Gaussian Processes for Machine Learning*, Cambridge: MIT Press. [420,421]

Reese, C. S., Wilson, A. G., Hamada, M., Martz, H. F., and Ryan, K. J. (2004), "Integrated Analysis of Computer and Physical Experiments," *Technometrics*, 46, 153–164. [418]

Robert, C., and Casella, G. (2004), *Monte Carlo Statistical Methods* (Springer Texts in Statistics) (2nd ed.), New York: Springer Verlag. [422,425]

Roberts, G. O., Gelman, A., and Gilks, W. R. (1997), "Weak Convergence and Optimal Scaling of Random Walk Metropolis Algorithms," *The Annals of Applied Probability*, 7, 110–120. [423]

Roustant, O., Ginsbourger, D., and Deville, Y. (2012), "DiceKriging, DiceOptim: Two R Packages for the Analysis of Computer Experiments by Kriging-based Metamodeling and Optimization," *Journal of Statistical Software*, 51, 1–55. [426]

Sacks, J., Welch, W. J., Mitchell, T. J., and Wynn, H. P. (1989), "Design and Analysis of Computer Experiments," *Statistical Science*, 4, 409–423. [418]

Santner, T. J., Williams, B. J., and Notz, W. I. (2003), *The Design and Analysis of Computer Experiments*, New York: Springer Verlag. [418]

Tuo, R., Wu, C. J., and Yu, D. (2014), "Surrogate Modelling of computer experiments with Different Mesh Densities," *Technometrics*, 56, 372–380. [418]

Xiong, S., Qian, P. Z. G., and Wu, J. C. F. (2013), "Sequential Design and Analysis of High-Accuracy and Low-Accuracy Computer Codes," *Technometrics*, 55, 37–46. [418]